



INTRODUCTION TO ADVANCED ROBOTICS

PROJECT 4

COPYING YOUR HAND

WHAT IS DIFFERENT ABOUT PROJECT 4

The first three projects all ended the same way. Look at the board, decide something, draw a box, stop. Do it once and you are finished.

This project has no finish. The camera looks at your hand, the arm moves to match it, the camera looks again, and that happens sixty times a second until you press q .

- *That is the whole difference, and it changes how you write the code. A one-shot program has one answer to get right. A control loop has to be right sixty times a second, forever, while a real arm is attached.*

This is the first project where the arm actually moves while you watch. In Projects 1 to 3 a bug meant a wrong number on the screen. In Project 4 a bug can mean the arm drives into a limit while you are standing next to it. Every safety habit in this module was built for this week.

A LOOP THAT NEVER FINISHES

Project 3 had a helper that captured frames in the background. This project does not need one. The loop is short enough to sit in the foreground, and the arm commands have to be issued from the same place as the decisions that cause them.

```
try:
    while cap.isOpened():
        ...                # look, decide, move, repeat

finally:
    cap.release()
    cv2.destroyAllWindows()
    mc.release_all_servos()
```

Read that `finally` carefully. There is no `except` in this program at all. It is not handling errors — it is guaranteeing that the arm lets go no matter how the program ends, including `Ctrl+C`.

That is the difference: an `except` swallows a mistake and carries on; a `finally` runs on the way out, always.

PUT THE ARM ON FIRST

Project 4 is the first project that will not run at all without the robot. Look at where the arm is created:

```
mc = MyCobot280('/dev/ttyAMA0', 1000000)
time.sleep(0.5)
```

That is line 16. It sits at the top level of the file, outside the `try` block, and outside any function. Nothing in it is a TODO. So it runs before you have written a single line of your own work.

- *No arm connected means no program. If the arm is off or unplugged you get a serial error at line 16 and you will never reach any of the code you actually came to write. Power the arm on, plug the cable in, then start typing.*

Compare Projects 1 to 3: the arm was created inside `main()` behind a `try/except`, so a missing robot printed a friendly message. Project 4 deliberately does not — it assumes real hardware is present.

A HAND IS 21 NUMBERS

MediaPipe is not OpenCV. In Project 3 you loaded a YOLO model and it handed back boxes and class names.

MediaPipe has a model built in, trained on hands, and it hands back 21 landmarks — one point per joint, from wrist to fingertip.

Every landmark carries three numbers:

.x how far across the picture, 0.0 at the left edge to 1.0 at the right edge

.y how far down the picture, 0.0 at the top to 1.0 at the bottom

.z roughly how far the joint is from the camera, in the same 0.0 to 1.0 units

The bundle of 21 is what MediaPipe calls a hand landmark set, and in this program it arrives as `results.multi_hand_landmarks` — a list of hands, each one a list of 21 landmarks. Two lists deep before you get to any real data.

NAMES INSTEAD OF COUNTING

You could reach the fingertip with `hand_landmarks.landmark[8]`. It works. It also means that if your memory of which finger is number 8 is wrong, the arm will do something baffling.

MediaPipe ships a list of names instead, and this project uses it:

```
index_tip = hand_landmarks.landmark[mp_hands.HandLandmark.INDEX_FINGER_TIP]  
thumb_tip = hand_landmarks.landmark[mp_hands.HandLandmark.THUMB_TIP]
```

`INDEX_FINGER_TIP` says exactly what it is. When the arm moves the wrong way in Project 4, this is the first place to look, because the alternative — a bare 8 — tells you nothing about which finger you actually grabbed.

Note the attribute name: `hand_landmarks.landmark` is singular even though it holds 21 of them. That is not a typo in your file, and it is not a typo in this slide. Typing `.landmarks` there is the single most common error in this project and the error message will tell you nothing helpful.

FRACTIONS, NOT PIXELS

This is the change that catches people who are comfortable with Project 3. In Project 3 the ArUco corners gave you pixel coordinates: 640 by 480, big numbers, easy to reason about.

MediaPipe gives you fractions of the picture instead.

`index_tip.x` of 0.5 means the fingertip is halfway across the frame, whatever resolution the camera happens to be running at.

- *This matters for two reasons. It works at any camera size, and it means you never have to divide by the frame width yourself. It also means a distance like the pinch threshold is a fraction of the picture, not a count of pixels — a fact you will want again in two slides.*

One consequence worth noticing now: because these are fractions rather than pixels, moving your hand towards the camera makes your fingers look further apart on screen even though your pinch is identical. Nothing has gone wrong. The measurement changed with the distance.

THE MIRROR

A camera sees you. You see the picture.

The two are opposite. If the picture is not flipped, moving your hand to your right moves it to the left of the screen, and the arm — which faithfully copies what it sees — moves to your left instead of your right. Everything is working perfectly and feels broken.

```
frame = cv2.flip(frame, 1)
```

The 1 means flip horizontally. No vertical flip, so up is still up.

This one line is the difference between the arm feeling like a mirror and feeling like it is arguing with you. If the arm responds but the wrong way along one axis, check this line before you check the map values — and remember this flip happens after the read and before the colour conversion, so everything downstream sees the flipped picture.

WRONG ORDER, WRONG COLOURS

Project 3 also converted colour, for the same reason. OpenCV reads pictures as blue, green, red. MediaPipe was trained on red, green, blue.

```
rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
results = hands.process(rgb_frame)
```

Two separate jobs on two lines, and the order is fixed. Convert first, then process the converted picture. You are also about to draw the skeleton back onto `frame`, which is still blue-green-red — so the drawing code later in the loop wants `frame` and the detection code here wants `rgb_frame`. Both names are in the file for a reason.

A swapped colour order does not crash. It quietly makes the model worse at its job, so this is the kind of mistake that looks like bad lighting rather than like a bug. If the skeleton is found but the detection is odd, swap the order back and watch what changes.

WATCHING BEATS LOOKING

MediaPipe can treat every frame as a fresh photograph, or as the next frame in a video. Project 4 asks for the second one:

```
hands = mp_hands.Hands(  
    static_image_mode=False,  
    max_num_hands=1,  
    min_detection_confidence=0.7,  
    min_tracking_confidence=0.7  
)
```

`static_image_mode` set to `False` means the first frame is expensive — it looks for a hand from scratch — and every frame after that is cheap, because it only has to follow the hand it already found.

This is the same idea as the threaded camera in Project 3, aimed at the model instead of the sensor. Switch it to `True` and the program still works, just several times slower, which for a program that has to keep up with a hand is not fast enough.

ONLY ONE HAND

`max_num_hands=1` is a safety limit wearing a disguise. It looks like a setting, and it is, but read it as a promise: this program will never be confused about whose hand it is copying.

The loop agrees. Even if MediaPipe did report two hands, the code walks through them one at a time and the last one wins — which would be a terrible way to control a real arm.

- *Two people in front of the camera is a genuinely dangerous situation for this program, and the setting is the only thing standing between them and a fight over who the arm is listening to. Keep it at 1.*

Both confidences sit at 0.7. Lower them and the program becomes twitchier and more likely to invent a hand that is not there; raise them and it stops tracking when your hand moves quickly. 0.7 is a reasonable middle, and it is a value you will be asked to justify in the debrief.

FROM FRACTIONS TO MILLIMETRES

The arm does not want 0.5. It wants 190 millimetres. This is the job of the mapping function, and it is the single idea holding this project together.

```
def map_value(value, in_min, in_max, out_min, out_max):  
    """Maps a value from one range to another and clamps it."""  
    clamped_val = max(in_min, min(in_max, value))  
    return out_min + (clamped_val - in_min) * (out_max - out_min) / (in_max - in_min)
```

Five arguments, and the last two are allowed to be backwards. Read the middle of the return line as how far through the input range am I, times the output range, plus the output minimum.

Because out_min and out_max can be in either order, the same function maps forwards and backwards. That is not a trick, it is just arithmetic — a negative range flips the direction. Step 10 needs both directions on the same axis.

THE CLAMP IS THE SAFETY

Look again at that function and find the line that is not arithmetic:

```
clamped_val = max(in_min, min(in_max, value))
```

It pulls `value` inside `[in_min, in_max]` before doing anything else. If a landmark ever reports 1.4 or -0.3 or something absurd, this function cannot pass that on.

- *This one line is what stops the arm being told to go somewhere it must never go. The workspace limits in this file are X between 130 and 250, Y between -160 and 160, and a fixed 180 of height. The clamp is what turns those four numbers into a guarantee rather than a suggestion.*

Notice the order inside: min first, then max. `min(in_max, value)` can only be at or below `in_max`; `max` lifts it back to `in_min`. Written the other way round, the two bounds would be supplied opposite.

ONLY THE MIDDLE SIXTY PERCENT

Now watch the numbers that get passed in during Step 10. The input range is not 0.0 to 1.0. It is 0.2 to 0.8.

- *So the top fifth and bottom fifth of your picture do nothing at all. Your hand has to be inside the middle sixty percent before the arm moves, and anywhere in that dead band the arm sits at its limit and stays there. This is deliberate: you cannot drive the arm to either end of its travel by accident.*

It also makes the control feel better. Without the dead band, the arm would creep very slowly near the top and bottom of the range, because small hand movements map to large arm movements when you are near a limit.

You can change 0.2 and 0.8 later and the arm will still work, which is exactly why they are worth writing down before you start. In the debrief you should be able to say what they buy you.

TWO POINTS MAKE A PINCH

A pinch has no key to press. It has a distance: thumb tip to index tip. That is all the information a pinch is.

```
def calculate_distance(p1, p2):  
    """Calculates 2D Euclidean distance between two MediaPipe landmarks."""  
    return math.hypot(p1.x - p2.x, p1.y - p2.y)
```

Two landmarks in, one number out. `math.hypot` is the straight-line distance between two points, and it saves you writing the square-root-of-the-squares yourself. The result is in fractions of the picture, not pixels — because the inputs were fractions.

Only two dimensions are used. The `z` value of each landmark is ignored, which means this measures how close the fingers look on screen rather than how close they are in the room. That is a deliberate simplification, and for a hand held roughly upright in front of a camera it behaves well.

ONE NUMBER DECIDES

The gesture itself is a comparison and nothing else:

```
pinch_distance = calculate_distance(thumb_tip, index_tip)
is_pinching = pinch_distance < 0.08 # Threshold for closed pinch
```

A boolean. Every later decision in this program hangs off `is_pinching` — the gripper command, the colour of the circle, and the colour of the text on screen. Set it once and three things follow.

0.08 is a fraction of the picture, so it is roughly eight percent of the frame. Because it is measured in fractions and not pixels, moving your hand closer to the camera shrinks the apparent distance and the same physical pinch stops registering — you have to pinch tighter. Keep your hand at a roughly constant distance from the camera and this behaves predictably.

SIX NUMBERS MOVE THE ARM

The arm is told where to go with one call carrying six values:

```
mc.send_coords([robot_x, robot_y, Z_HEIGHT, -180, 0, -135], ARM_SPEED, 0)
```

X, Y and Z are millimetres from the base of the arm. The other three are degrees of wrist rotation, and in this program `they never change`. Only two numbers in the whole list are ever anything else.

The last argument is 0, meaning move there now.

`ARM_SPEED` of 50 out of 100 is a deliberate middle — fast enough to feel responsive, slow enough that a mistake does not become an incident.

Because X, Y and Z arrive as a list of plain numbers, the arm is given an absolute destination rather than a nudge. There is no memory between commands — move your hand and the next command describes the new place entirely.

OPEN AND CLOSED

The gripper is two commands, both inside the rate-limited block:

```
if is_pinching:
    mc.set_gripper_state(1, 80) # Close gripper / activate pump
    status_text = "Pinch Active (GRIP)"
else:
    mc.set_gripper_state(0, 80) # Open gripper / deactivate pump
    status_text = "Tracking (OPEN)"
```

The first argument is the state — 1 for closed, 0 for open. The second is a speed. The comments mention a pump because the same command drives one, and reading the comments is how you know that.

Notice that this block sends a gripper command every time the rate limit allows, whether or not the pinch state has changed. Sending close twenty times when you have not moved is wasteful, but it is also simple and it is safe: the arm cannot be left gripping by accident if you shake your hand open. Simplicity wins here.

THE ARM IS SLOWER THAN THE CAMERA

The camera produces about thirty frames a second. The serial cable to the arm is far slower than that, and if you ask it for a move on every single frame the queue behind it grows until the program stalls.

```
last_command_time = 0  
COMMAND_INTERVAL = 0.15  # Send move command every 150ms
```

A tenth of a second between commands means the arm is told where to go at most six or seven times a second, while the picture is still examined thirty times a second.

- *This is the fix for a problem you have not hit yet, and it is the reason the program feels smooth rather than twitchy. Notice also where the telemetry drawing sits — outside this rate limit. Drawing is free; moving the arm is not.*

RATE LIMITING, HONESTLY

Here is the whole mechanism:

```
current_time = time.time()
if current_time - last_command_time > COMMAND_INTERVAL:
    # Send position command: [X, Y, Z, Rx, Ry, Rz]
    print(f"Target: X={robot_x:.1f}, Y={robot_y:.1f}, Z={Z_HEIGHT}")
    mc.send_coords([robot_x, robot_y, Z_HEIGHT, -180, 0, -135], ARM_SPEED, 0)

    # Control end-effector based on pinch state
    if is_pinching:
        mc.set_gripper_state(1, 80) # Close gripper / activate pump
        status_text = "Pinch Active (GRIP)"
    else:
        mc.set_gripper_state(0, 80) # Open gripper / deactivate pump
        status_text = "Tracking (OPEN)"

    last_command_time = current_time
```

Compare, act, then update the record. On the very first frame the stored time is 0, so the difference is enormous and the block always runs at least once.

status_text looks like a bug: assigned inside the if block, read far below it. It works because the block runs on the first frame, so the name always exists from then on.

LETTING GO AT THE END

The last three lines of the file are the ones that matter when something has gone wrong:

```
finally:  
    cap.release()  
    cv2.destroyAllWindows()  
    mc.release_all_servos()
```

`cap.release()` hands the camera back so the next program can have it. `destroyAllWindows()` closes the window. And `release_all_servos()` lets the arm go limp.

- *Limp means you can move the arm by hand afterwards, and it means the motors are not holding position against your fingers while you reach in to unplug something. If the servos buzz or the arm will not budge after the program exits, this line has been deleted or skipped.*

Because it is a finally block, it runs even when the program crashes — the entire reason the cleanup lines live here.

WHAT COULD ACTUALLY HURT SOMEONE

Before any code, the list of things standing between this program and an injury. Every one of them is a decision already made for you:

- ☐ The mapping is clamped to 0.2-0.8 before it becomes millimetres.
- ☐ The arm is confined to X 130-250, Y -160-160, and a fixed height of 180.
- ☐ Only one hand is ever listened to.
- ☐ The arm moves at half speed, six times a second, never faster.
- ☐ If the hand leaves the picture the arm is not given any new instruction at all.
- ☐ And if you stop the program, the servos are released.

STEP 1

The module's standard import block, with two additions for this project:

```
import cv2
import mediapipe as mp
import math
import time
import numpy as np
```

`cv2` is the camera, the drawing and the colour conversion.
`math` is for one function. `time` does the rate limiting.
`mediapipe` is the new one.

`numpy` is in this list and is not used anywhere in Project 4 — not once. It is in the import block of every project in this module, and it earns its place in Projects 1, 2 and 3. Here it does nothing. That is harmless, and it is also a clue: if you are wondering why your frame shape behaves as an object rather than a list, this is where that comes from.

STEP 2

The arm, exactly as in the previous three projects:

```
from pymycobot.mycobot280 import MyCobot280
```

Then the two lines of setup that are not a TODO but which you should read carefully anyway:

```
mc = MyCobot280('/dev/ttyAMA0', 1000000)
time.sleep(0.5)

X_MIN, X_MAX = 130, 250    # Forward / Backward range
Y_MIN, Y_MAX = -160, 160   # Left / Right range
Z_HEIGHT = 180             # Fixed height above surface (mm)
ARM_SPEED = 50             # Movement speed (1-100)
```

The workspace numbers sit at the top so every later calculation uses them once. The half-second sleep lets the arm finish booting after the serial port opens — a command to a still-starting robot is a classic lost first command.

COPY IT, AND START IT ONCE

One file this time — but the arm must be powered on and plugged in before you run anything:

```
scp M2-P4-Starter.py er@192.168.1.149:Documents
```

```
python M2-P4-Starter.py
```

Run it now, with every vision TODO still empty. Expect this:

```
Traceback (most recent call last):  
  File "M2-P4-Starter.py", line 63, in <module>  
    while cap.isOpened():  
NameError: name 'cap' is not defined
```

The camera TODO stayed empty, so `cap` was never created and the loop asked for it anyway.

A serial error at line 16 means the arm is not there. Power it on and run again before changing any code.

STEP 3

Two lines to name the parts of MediaPipe you will use, then the model itself:

```
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils

hands = mp_hands.Hands(
    static_image_mode=False,
    max_num_hands=1,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.7
)
```

The first two lines are shortcuts so the rest of the file can say `mp_hands` instead of `mp.solutions.hands`. This is the first project in the module to use MediaPipe, and Project 6 will use it again the same way.

STEP 4

The camera, the same three lines as Project 3:

```
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
```

Nothing to think about here beyond one point: the resolution you ask for is not a promise. Some cameras give you 640 by 480, some ignore the request, and that is fine.

It is fine because everything downstream works in fractions. MediaPipe does not care what the frame is called, and the code never multiplies a landmark by a width.

That is the payoff for the choice on slide eight. In Project 3 you had to know the pixel size, because the detector returned pixel coordinates. Here you never do, so this step cannot be the reason the program fails later.

STEP 5

The mapping function, the whole of it:

```
def map_value(value, in_min, in_max, out_min, out_max):  
    """Maps a value from one range to another and clamps it."""  
    clamped_val = max(in_min, min(in_max, value))  
    return out_min + (clamped_val - in_min) * (out_max - out_min) / (in_max - in_min)
```

Do not simplify the return line. The whole expression is one line in the file and it is worth typing exactly.

If you want to be sure it is right, test it away from the robot:

`map_value(0.5, 0.2, 0.8, 130, 250)` is 190, the middle of the range.

`map_value(0.9, 0.2, 0.8, 130, 250)` is still 250, because of the clamp.

Those two numbers are worth being able to produce on request.

STEP 6

The distance function. Three lines, and the middle one is a docstring that you should keep:

```
def calculate_distance(p1, p2):  
    """Calculates 2D Euclidean distance between two MediaPipe landmarks."""  
    return math.hypot(p1.x - p2.x, p1.y - p2.y)
```

This is the first project where a helper takes MediaPipe objects as arguments. It works because a landmark has `.x` and `.y` on it, so this function never needs to know what it is measuring.

Two independent guesses about what the arguments are will both fail loudly and differently: forget the parentheses and you get a `TypeError` about missing arguments; pass two whole hands instead of two landmarks and you get an `AttributeError` about `.x`. Both messages point straight at this function.

STEP 7

Reading the frame, flipping it, converting it, and asking MediaPipe. Five lines, in that order:

```
success, frame = cap.read()
if not success:
    continue

# Flip horizontally for intuitive mirror view
frame = cv2.flip(frame, 1)
h, w, _ = frame.shape
rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
results = hands.process(rgb_frame)
```

The `continue` matters more than it looks: it skips the rest of the loop body and goes straight back for another frame, rather than carrying on with a frame that was never filled in.

`h, w, _ = frame.shape` is the Project 1 unpacking, read [here](#) because it is the first line in the loop with a frame to measure.

STEP 8

Draw the skeleton. One line, and it draws onto `frame` — not onto `rgb_frame`:

```
mp_drawing.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)
```

Three arguments: the image to draw on, the landmarks, and the list of pairs describing which joints connect to which. The connections list is part of MediaPipe, which is why it is reached through `mp_hands` rather than being written out here.

This line runs before the landmarks are extracted on the next step, and that is fine — drawing does not need to know which two joints you are about to use. Read the loop as: draw everything, then use what you need. If the skeleton appears but the circle does not, this step is done and Step 13 is not.

STEP 9

Now pick out the two joints the rest of the program cares about:

```
index_tip = hand_landmarks.landmark[mp_hands.HandLandmark.INDEX_FINGER_TIP]  
thumb_tip = hand_landmarks.landmark[mp_hands.HandLandmark.THUMB_TIP]
```

Two names, and from here on the program stops thinking about hands entirely. It has an `index_tip` and a `thumb_tip`, and it never needs the other nineteen landmarks again.

Two of twenty-one landmarks is all this project uses. Project 6 counts fingers instead, and needs rather more of them. It is worth noticing how little of the model output Project 4 actually touches — the hard part was never the landmarks, it was the millimetres.

STEP 10

The heart of the project. Two lines, one per axis:

```
robot_x = map_value(index_tip.y, 0.2, 0.8, X_MAX, X_MIN)
# Vertical hand position -> Robot Forward/Back
robot_y = map_value(index_tip.x, 0.2, 0.8, Y_MIN, Y_MAX)
# Horizontal hand position -> Robot Left/Right
```

Both lines are wrapped here so they fit the screen — in your file each is one long line with its comment at the end. Every character is the same.

Look at the `x` line carefully. The finger's **height** sets how far forward the arm reaches, and the output range is given as `X_MAX, X_MIN` — backwards.

Backwards on purpose. `y` is small at the top of the picture, so raising your hand maps to `X_MAX` and the arm reaches forward. Combined with the mirror flip, that is the behaviour you want; if the arm goes the other way, swap those two arguments.

STEP 11

The pinch. Call the function you wrote, then compare:

```
pinch_distance = calculate_distance(thumb_tip, index_tip)
is_pinching = pinch_distance < 0.08  # Threshold for closed pinch
```

Nothing here is new — Step 6 built the measurement and slide seventeen explained the comparison. What is new is that this boolean now decides whether the gripper closes.

Change 0.08 too early and you will spend an afternoon convinced the gripper is broken. Leave it exactly as it is until the whole program runs, then tune it once with the telemetry on screen. There is a slide on the cost of moving it near the end.

STEP 12

The longest block in the project, and the one that moves the arm:

```
current_time = time.time()
if current_time - last_command_time > COMMAND_INTERVAL:
    # Send position command: [X, Y, Z, Rx, Ry, Rz]
    print(f"Target: X={robot_x:.1f}, Y={robot_y:.1f}, Z={Z_HEIGHT}")
    mc.send_coords([robot_x, robot_y, Z_HEIGHT, -180, 0, -135], ARM_SPEED, 0)

    # Control end-effector based on pinch state
    if is_pinching:
        mc.set_gripper_state(1, 80) # Close gripper / activate pump
        status_text = "Pinch Active (GRIP)"
    else:
        mc.set_gripper_state(0, 80) # Open gripper / deactivate pump
        status_text = "Tracking (OPEN)"

    last_command_time = current_time
```

Two details to get right. Everything that touches the arm sits inside the `if`, including the gripper calls and the assignment to `last_command_time`. And the `print` comes before the move, so if the arm is not moving at all but numbers are appearing in the terminal, the program has already calculated a target and failed to deliver it — which is a different fault from producing no numbers at all.

STEP 13

Telemetry, part one: the circle, and the position readout.

```
cx, cy = int(index_tip.x * w), int(index_tip.y * h)
cv2.circle(frame, (cx, cy), 10, (0, 255, 0) if not is_pinching else (0, 0, 255), -1)

telemetry = f"Robot Target -> X: {int(robot_x)}mm | Y: {int(robot_y)}mm"
cv2.putText(frame, telemetry, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
```

Here the fractions finally become pixels. `index_tip.x * w` turns "halfway across" into an actual column number, which is the only kind of thing `cv2.circle` can use. Green circle means open, red means pinching — the same condition as the gripper, read one more time.

This whole block sits outside the rate limit on purpose, so the overlay updates smoothly even though the arm is commanded only seven times a second. The screen rounds with `int()` and the terminal prints one decimal — neither is wrong; do not be surprised they differ.

STEP 13, CONTINUED

And the state readout, which is the longest line in the project:

```
cv2.putText(frame, f"State: {status_text}", (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.6,  
(0, 255, 0) if not is_pinching else (0, 0, 255), 2)
```

Wrapped across two lines here for the same reason as Step 10 — in the file it is a single line of 152 characters. The colour is chosen the same way as the circle: green while open, red while pinching.

This is the third and last read of `is_pinching` in one loop: the gripper command, the circle colour, and now this text. One boolean, three consequences. If you ever find yourself writing the pinch test a fourth time, something has gone wrong with your plan rather than your typing.

STEP 14

The last TODO. Show the picture:

```
cv2.imshow("Mission 2 - Project 04: Hand Control", frame)
```

That is the whole of it. The exit check was already written for you:

```
if cv2.waitKey(1) & 0xFF == ord('q'):  
    break
```

The window title is not decoration. It is how you know which program has the camera, and in a lab where four of these are running it is the only honest answer to "why can I not see anything".

Wait, then show, then go round again. The order matters: `waitKey(1)` keeps the window responsive, `imshow` puts the picture in it, and the loop condition re-checks that the camera is still open. Press `q` and the loop breaks, the finally block runs, and the arm lets go.

READING THE SCREEN

Four things on the window at once, and each one tells you something:

- **The skeleton** — MediaPipe found a hand. No skeleton means the problem is lighting, distance, or confidence, not your mapping code.
- **The circle** — green while the gripper is open, red while it is closed. If it never turns red, your pinch threshold or your hand distance is wrong.
- **X and Y in millimetres** — the numbers the arm was last told. They should move as you move your hand, and they should never leave 130-250 or -160-160.
- **The state line** — the same pinch, in words.

Read them in that order when something is wrong. No skeleton rules out every step after Step 9. A skeleton but a still circle rules out Steps 10 and 11. Numbers on screen but a motionless arm means Step 12.

BEFORE YOU TUNE ANYTHING

What you see	What it means	What to do
Serial error at line 16	The arm is not connected	Power it on, check the cable, run again
NameError: name 'cap'	Camera TODO still empty	Finish Step 4 before debugging
No window at all	Program died before imshow	Read the traceback from the bottom up
No skeleton on screen	No hand found in frame	Check lighting and hand distance
Skeleton, circle never red	Pinch not reaching 0.08	Tighten the pinch, move the hand closer
Arm moves the wrong way	Axis inverted on one direction	Swap X_MAX and X_MIN, not the function
Arm will not budge afterwards	Servos never released	Put release_all_servos() back

Two of these are not really faults. A silent program with no window is a TODO you have not filled in yet, and an arm that will not move by hand is the servos doing exactly what they were told. Everything else in that table is a real fault, and each one points at a single step.

WHAT THE THRESHOLDS ACTUALLY COST

Project 3 had three thresholds and no way to reason about them. You have four, and you can now say what each one trades away.

- 1 0.7 detection confidence**
too low and it invents hands that are not there
- 2 0.2 and 0.8 mapping range**
too wide and the arm can be driven to either limit by accident
- 3 0.08 pinch distance**
too tight and the gripper will not close on a real object
- 4 0.15 command interval**
too tight and the arm stutters or stops responding entirely

Every one of them is a trade, not a setting to maximise. A threshold is a decision about which mistake you would rather make — different for a gripper that must not drop something and a display that must not flicker.

CHALLENGE
YOUR MOVE

THE BRIEF

Make the arm copy your hand.

Move your hand and the arm follows. Pinch and the gripper closes. Keep it smooth, keep it safe, and keep it inside the workspace.

Every decision is already made for you in the constants at the top of the file. Your job is not to choose safety — it is to understand why it was chosen, well enough to explain it.



- ☐ The arm tracks a single hand, in two axes, at a fixed height.
- ☐ The gripper responds to a pinch, and nothing else.
- ☐ The arm never leaves the workspace, whatever your hand does.
- ☐ The servos are released when you stop.

YOUR MISSION: SMOOTH OR IT DOES NOT COUNT

A control loop is judged on how it behaves while nothing is wrong. Get these four and the project is finished:

- 1 Move your hand left. The arm moves left.**
Not right. If it is inverted, fix the map, not the flip.
- 2 Raise your hand. The arm reaches forward.**
X_MAX and X_MIN in that order, on purpose.
- 3 Pinch. The gripper closes. Open. It releases.**
Green circle to red circle, no drift in between.
- 4 Move fast. The arm does not stutter.**
If it judders, COMMAND_INTERVAL is too small.

● *Take your hand out of frame. The arm must stay exactly where it was and say nothing new. If it keeps moving, you have given it an instruction during a frame with no hand in it.*

YOUR DATA LOG

Fill this in before you change any threshold. Every row should be a measurement, not a guess.

Where your hand is	X commanded	Y commanded	Gripper	Did it feel right?
Low and centred	_____	_____	_____	_____
Middle and centred	_____	_____	_____	_____
High and centred	_____	_____	_____	_____
Middle and far left	_____	_____	_____	_____
Middle and far right	_____	_____	_____	_____
Pinched, hand still	_____	_____	_____	_____
Hand leaves the frame	_____	_____	_____	_____

- *That last row is the one to get right. Write what the arm actually did, not what you hoped it did. If the X and Y cells in that row contain new numbers, your program is still instructing the arm with no hand in view.*

DEFINITION OF DONE

Tick every line. A project is not finished because the window opens.

- ☐ All 14 TODOs filled in, with the comments copied as written.
- ☐ Every threshold in the file explained in one sentence each.
- ☐ Data log complete for all seven rows.
- ☐ Hand leaves frame three times: arm holds position, every time.
- ☐ q exits cleanly, window closes, servos released, arm moves by hand.
- ☐ One person's hand at a time. No exceptions during testing.

The second line is the one that separates someone who ran this once from someone who understands it. The first four are only finished when you can say what each number is for without looking.

GO FURTHER

Each of these is a small change to code you have already written.

- **Smooth it.** Average the last few `index_tip` values before mapping them, and watch the arm stop trembling. A short list and a `sum()` will do it.
- **Two hands.** Raise `max_num_hands` and average both index tips. Now decide what should happen when they disagree.
- **Fingers instead of pinch.** Count how many fingertips sit above the wrist and use that as the gripper command. You already have all 21 landmarks.
- **Record and replay.** Log the X and Y values to a file, then drive the arm from the file instead of from the camera.
- **Make it stop safely.** Have the program return the arm to a home position when it loses the hand, rather than holding still forever.

The first one is the most valuable and the least dramatic. Most of the work is deciding how much of the noise to believe.

MISSION DEBRIEF